Journal of Real-Time Image Processing (2023) 20:75
https://doi.org/10.1007/s11554-023-01324-5

RESEARCH

Hardware acceleration of YOLOv7-tiny using high-level synthesis tools

Adib Hosseiny[1] · **Hadi Jahanirad**[1]

Received: 17 March 2023 / Accepted: 29 May 2023 / Published online: 17 June 2023
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2023

Abstract
FPGAs have emerged as a promising platform for implementing neural networks due to their reconfigurability, parallelism, and low power consumption. Nonetheless, designing and optimizing FPGA-based neural network accelerators is a complex and time-consuming task with register transfer level (RTL) languages. High-level synthesis (HLS) tools provide a higher level of abstraction for FPGA design, enabling designers to concentrate on top-level design aspects, such as algorithms, rather than low-level hardware implementation details. One of the state-of-the-art object detection networks is you look only once (YOLO) network series which is constructed using different neural network technologies using cross-stage connections and feature extraction techniques like pyramid networks. In this paper, we propose a method for the implementation of YOLOv7-tiny network on FPGAs using HLS tools. We present a comprehensive analysis of the performance and resource utilization of FPGA-based neural network accelerators. Our methods show excellent results for real-time application requirements such as latency. Specifically, our work reduces the usage of digital signal processing (DSP) units by 90% and it saves up to 60% of flip-flops compared to state-of-the-art designs, while achieving competitive usage of block RAM and look-up tables. Additionally, the achieved design latency of 15 ms is extremely suitable for real-time applications. Also we will propose a method for BRAM utilization method and off-chip memory access.

Keywords High level synthesis · Convolutional neural network · Object detection · FPGA · YOLO

1 Introduction

For accelerating neural networks on FPGAs, HLS tools can be used to generate designs that perform computations for various neural network layers such as convolutional layers, concatenation layers, and pooling layers. These designs can be optimized for speed and resource usage, enabling efficient execution of neural networks on FPGAs.

There are plenty of object detection algorithms that are used in object detection like R-CNN (region-based convolutional neural networks), you only look once (YOLO), spatial pyramid pooling network (SPP) [1], and other R-CNN-based methods such as Fast R-CNN [2] and modified versions like faster R-CNN [3]. Unlike all other object detection methods, YOLO uses a single CNN to predict bounding boxes around objects and their probability of being an object and their target class.

Numerous studies have endeavored to incorporate diverse object detection and deep learning algorithms on embedded platforms, specifically FPGAs [4–10]. According to [11], the YOLO series are the most suitable networks for real-time object detection algorithms among other algorithms. Other algorithms cannot be implemented in FPGAs as easily as YOLO is implemented because there are lots of recursive calls to functions inside the network that makes them computationally expensive in terms of memory management and handling huge memory requirements of those architectures.

2 Related works

There are lots of read/write operations and memory accesses during the neural network inference stage. According to [12], an ideal read/write operation is that the synthesized RTL can start writing exactly after reading operations, and never need to pause for write permissions. CNNs exhibit a sequential characteristic, whereby each input feature map slice proceeds to a

✉ Hadi Jahanirad
h.jahanirad@uok.ac.ir

Adib Hosseiny
adib.hosseiny@uok.ac.ir

[1] Department of Electronics and Communication Engineering, University of Kurdistan, Sanandaj 90210, Iran

layer for computations, and the resulting output serves as input to the subsequent layer. This nature of CNNs, characterized by a sequential behavior, is intrinsic to their design. However, Nguyen et al. [11] employs a streaming data approach that determines when the sliding cube within the network will traverse a given layer and when the layer will accept new inputs.

To construct such hardware, we must consider the latencies of different layers of the network and make sure that each layer's requirement (latency, resource usage, etc.) is met. The proposed method in [13] uses fixed-point implementation for the acceleration of the tiny YOLOv2 algorithm. The implementation of this approach has the potential to decrease resource utilization. However, it is important to note that this comes at the cost of sacrificing a certain degree of algorithmic accuracy. Other implementations like the one in [14] use quantization for activation function outputs and convolutional layer computation. The proposed architecture in [13] implements YOLOv2-tiny that proposes a design using general matrix multiplication (GeMM) core that is designed in an OpenCL environment and achieves 3.06 fps for tiny-YOLOv2 which cannot satisfy real-time inference requirements.

This quantization makes the network much faster and more resource-efficient than the networks that use word lengths like a 16-bit or 32-bit floating point data type. Although quantized neural networks can save a lot of resources and even be faster; you need to do some modifications in your training phase also and even train with quantized values for activation functions and whatever the layers are. Because whatever the structure of the network is, it is fixed during the inference stage and can lead to an accuracy loss if not well-designed.

According to [15], using kernel size padding helps to just implement the maximum kernel size. Then the computation of smaller kernel sizes is done by embedding the maximum kernel size. Also, this work uses layer fusion computation as shown in Fig. 1. It is a strategy to fuse every N layer as one layer that is named layer batch and then that layer batch provides information to the network that its calculations are done or not for being fed by new inputs.

The architecture used in [16], implements YOLOv4 on Xilinx Zynq FPGAs. This work optimizes frame rate processing using a method that drops a frame if the delay for an image processing to the output is bigger than the streaming delay (which is ~ 33 ms @ 30 fps input stream). Also, this work has been trained on a custom dataset with 91 classes with 2.5 million labeled instances in 328 k images. The best latency for this method is 10 ms and the worst is 18 ms at a 30 fps input image rate. Which dedicates much higher time for one frame processing than the actual maximum latency for the accelerator to work.

However, this implementation can increase frame rate up to the level that satisfies 30 fps with frame dropping when programmable logic (PL) is not responding; but their best results are at a 30 fps input rate.

3 About YOLO

The first version of YOLO had much fewer computational layers, including convolutional and pool layers. But as the backbone of YOLO changed in YOLOv3[17] from Darknet-19 to a residual-based Darknet-53 there were problems appeared to be solved by extending the convolutional layers and chaining 3×3 convolutional layers with 1×1 convolutional layers and also adding residual blocks after 3×3 convolutional layers to add the result of the previous convolutional layer with the result of a residual block by using α and β factors that determine how each adding operand should take effect for better predicting bounding boxes and improving accuracy. The structure of the residual block in DarkNet-53 is demonstrated in Fig. 2.

In Yolov4 [18] the backbone changed from darknet-53 to CSPDarknet-53 which includes a new technology in the backbone of the YOLO series that is CSPNet [19] that is derived from DenseNet [20]. DenseNet is concatenating all of the feature maps from all layers to subsequent layers. This architecture brings the functionalities of DenseNet with modifications like distributing feature maps among dense and main connections. This contribution adds on top of computational complexity but also gains attractive results using it.

The YOLOv4 framework contributed to the field of object detection by incorporating various tools and technologies such as Data Augmentation, SE [21], SPP [1], SAM [22], and PAN [23] referred to as BoF (Bag of Freebies) and BoS (Bag of Specials). BoFs are primarily relevant to the training

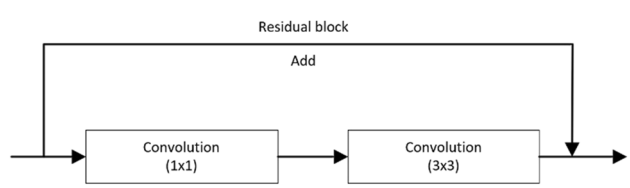

Fig. 2 Residual block with $\alpha = \beta = 1$

Fig. 1 Layer fusion computation

process, whereas BoSs are not related to the training strategy. Specifically, in the context of real-time inference on FPGA, attention is given to the BoSs to understand their impact and the role they play in improving the algorithm. Although they only affect the inference cost by approximately 1%, they significantly improve the accuracy.

YOLOv7 [24] comes with an extended set of BoFs and BoSs to improve the accuracy of the older versions. The main contributions of YOLOv7 with respect to taking place in the training process. The YOLOv7 BoSs includes SPP, SE, SAM, FPN [25], and non-maximum suppression (NMS). Feature pyramid network (FPN) is a feature map structure with an f-level pyramid, there are feature maps with different sizes coming from different layers that concatenate at the next layer of the network.

During the training phase, these feature maps from different pyramids are going to the head section of the network containing convolution layers with different kernels and then calculate the loss and back-propagating to learn the parameters of the network. Given bounding boxes around objects in the output of the network, there are several bounding box predictions around an object. By using NMS, the network can ignore those bounding boxes that are not well-confirmed to be an object.

Exporting the open neural network exchange (ONNX) model file in the official YOLOv7 repository, some options are used to generate the desired model file in a way that our implementation should work. For example, *iou_threshhold* is a parameter that defines the distance of the original object and the detected one that is acceptable in output for being an object. One-stage detectors like YOLO, detect these coordinates and probabilities in one execution of the network. *conf_thresh* is another parameter passed to the export function to accept only objects that we are at least *conf_thresh* percent confident of being an object. The overall structure of YOLO network family is shown in Fig. 3.

Scaled versions of the YOLO series do some modifications to the number of layers. Tiny-YOLOv7 also is a scaled version of the YOLOv7 network. The most important difference is the number of convolution layers and pooling layers. The YOLOv7 network has 306 layers while the tiny version has 200. The other difference is the activation functions. YOLOv7 uses a sigmoid activation function and tiny has a Leaky-Relu activation function. Comparisons between the two ONNX-exported models are demonstrated in Table 1.

4 Proposed method

ONNX-exported YOLOv7-tiny consists of 35(1 × 1) convolutional layers and 23 (3 × 3) convolutional layers, which sums up to 55 convolutional layers in total. It has six pooling layers of type max pooling with different strides and paddings. Followed by each convolutional layer, there is an activation function of the Leaky-Relu type with a value of 0.10000000149011612. The network contains two up-scaling layers for up-sampling stuff and, 14 concatenation layers, each of which concatenates some of the preceding layers. Three reshape layers break the output matrix's dimensions of the previous layer into 2 dimensions to get class outputs and probabilities. Finally, three transpose layers are utilized to transpose the related matrix to get the desired format. An overview of our proposed design process for YOLOv7-tiny is demonstrated in Figs. 4, 5.

Activation, Transpose and Reshape layers do not contain learnable parameters. They do a mathematical function on the received feature map. However, they would be considered for total latency calculation in the HLS code. Numbers of available various types of layers in the exported YOLOv7-tiny ONNX model are displayed in Table 2.

After exporting ONNX, the first step of the design is to simulate the network that is written in HLS-C++ and get results back to ensure that the network works properly. When the simulation is passed in Vitis-HLS, the next step is to synthesize the HLS code to generate our desired hardware. This step is the most complex stage of our implementation because of the scale of our network and the large number of layers that should be processed sequentially to produce an output matrix. The output matrix corresponds to detection

Table 1 Comparison of YOLOv7 and YOLOv7-tiny

Network	Layers	Parameters	Gradients
YOLOv7	310	36,905,341	36,905,341
YOLOv7-tiny	141	6,219,709	6,219,709

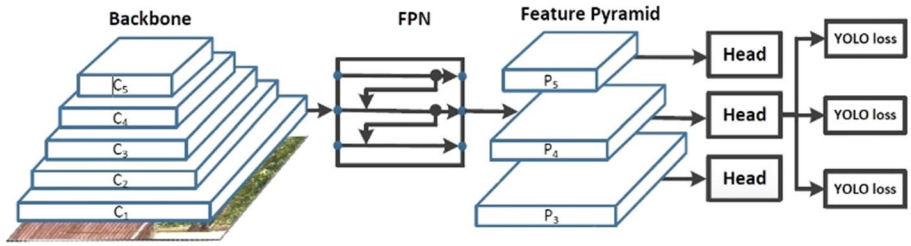

Fig. 3 YOLO-v3 architecture presented in [26]

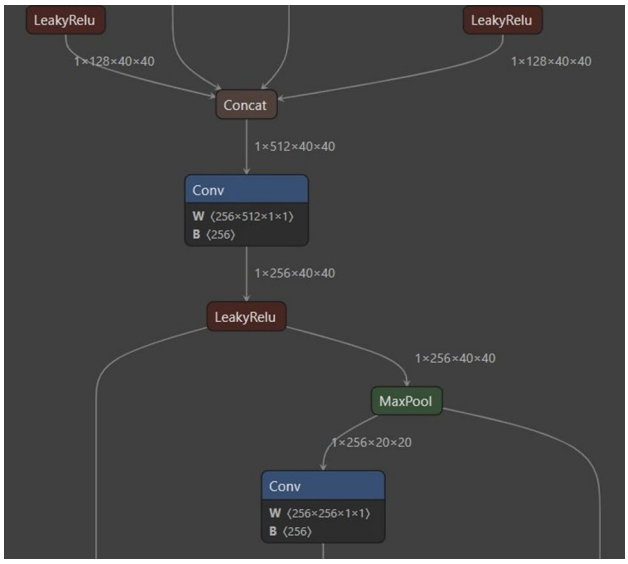

Fig. 4 A chunk of YOLOv7-tiny ONNX model

of 80 objects and determining their locations in the input image.

5 Model extraction

First, the network is trained using python and the official YOLOv7 repository. The output model file is a.pt file (Pytorch model extension file) containing all the trained model's properties and data. Various versions of the YOLO network such as YOLOv7-x and YOLOv7-d6, YOLOv7-e6, YOLOv7-tiny, or YOLOv7-e6e could be trained similarly. Next, the output Pytorch file is exported as an ONNX file to easily view all the model properties. For this purpose, we use Netron (a viewer for neural networks, deep learning, and machine learning models). An example of viewing the model file in Netron is in Fig. 4.

6 Design network in HLS

Convolutional layers have different dimensions regarding their inputs, weights and, biases. Moreover, convolutional layers have different properties like padding, dilation, stride, and kernel size. All these features are configured in the network structure in the training stage and their corresponding values are in the corresponding model file. An overview of design techniques used for each section in the network is illustrated in Table 3.

For being efficient in convolutional layer design, HLS considerations like resource partitioning, loop optimizations, and reducing loop sizes according to padding and stride sizes should be done. These optimizations reduce the total latency of the design by a huge amount according to Eq. (1). The output feature map of convolutional layers goes into the activation layer for avoiding the gradient vanishing problem.

$$\text{latency}_{\text{total}} \approx \text{latency}_{\text{conv}} + \text{latency}_{\text{pool}} \tag{1}$$

A meaningful amount of the design latency is imposed by pooling layers in the network. Some of the pooling layers in YOLOv7-tiny can be run concurrently because of the same input feature map. Our utilized techniques to implement pooling layers in HLS include shifting and taking the absolute value of the results, defining helper functions like the max function and, using "*hls_math*" library functions. Moreover, we utilize the mentioned techniques of Table 3 to optimize the implemented pooling layers. The output shape of the pooling layer with padding (P) and stride (S) and kernel size (K) is shown in Eq. (2).

$$\text{out}_{\text{shape}} = \frac{\text{in}_{\text{shape}} - K + 2\times P}{S} + 1 \tag{2}$$

The concatenation layer must concatenate all input layers into one layer. The difference between summation and

Table 2 Number of layers in YOLOv7-tiny

Layer	#Count
1×1 convolution	35
3×3 convolution	23
Max pooling	6
Concatenation	14
Reshape	3
Transpose	3
Activation	55
Up-scaling	2

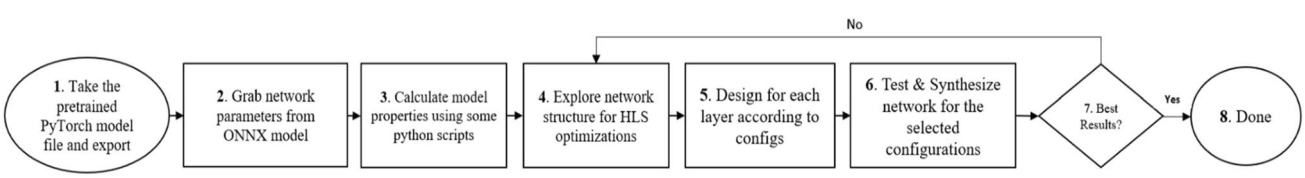

Fig. 5 Proposed design for YOLOv7-tiny accelerator using HLS tools

Table 3 Optimizations used in the HLS design

Network section	Optimizations
Convolutional, activation layer	1. Combining activation function with bias loop 2. Invert kernel loops by using *abs()* function for non-zero padding 3. Pipeline assignment loops like bias with II = 1 4. Unroll computation loops like kernel with different factors based on loop size
Pooling layer	1. Invert filter loops with *abs()* 2. Pipeline assignment loops if II = 1 possible; if not unroll with tiny factors (because bigger) 3. Change filter loop optimizations according to layer properties like stride and padding
Up-sampling layer	1. Unroll with different factors 2. Pipeline assignment loops with II = 1
Concatenation layer	1. Concatenation method in Fig. 7
Reshaping layer	1. Pipeline assignment operation with II = 1
Special regions in the network such as parallel layers and identity connections between layers	1. Allocate same tensor for parallel layers with no WAR/RAW conflict 2. Separate tensors for each parallel layer 3. Reuse tensors in later layers of the network

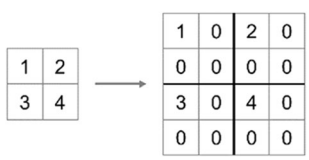

Fig. 6 Up-scaling layer with zero padding

concatenation is that in concatenation we just put every input feature map behind each other that resulting in the output feature map being of larger size, but in the summation layer each corresponding value is summed in all input feature maps and results in the same size of output feature maps.

Consequently, unlike summation layers, there is no addition and calculation in the concatenation layer. HLS compiler attempts to address array partitions which we want to concatenate them. The best choice for this situation is to chain the pointer addresses by each array partition in the input of the concatenation layer. During synthesis, Vitis-HLS partitions the arrays by allocating the result to an output array and then chaining the base address of input arrays to that output array by using offsets (input array sizes), concatenation is done within HLS. Figures 6, 7 explains compilation of the concatenation layer with 3 inputs in HLS.

Up-scaling layer somehow reverts the pooling layers as it is a layer that assigns 2 proximity values of the input feature map to the same output value in the output feature map with a definition of stride = 2. Up-scaling layer in the network is the floor type resizing wherein the minimum value of the corresponding input value would be kept. This operation is also called up-sampling. Figure 6 shows the functionality of up-scaling layers in the YOLOv7-tiny network with zero padding.

7 Network properties

Yolov7-tiny like other neural networks has a set of learned parameters (weights and biases) that are tuned during training phase using a specific dataset. Convolutional layers contain such learnable parameters. On the other hand, pooling and resizing layers do not contribute any learnable parameters. These layers just receive an input feature map and then apply some mathematical functions like adding and subtracting to them, and then they produce the output feature map. Yolov7-tiny has trained on MS-COCO [27] dataset and thus it has to predict 80 classes.

The YOLOv7-tiny network features distinct regions where it is possible to reuse a single tensor for storing

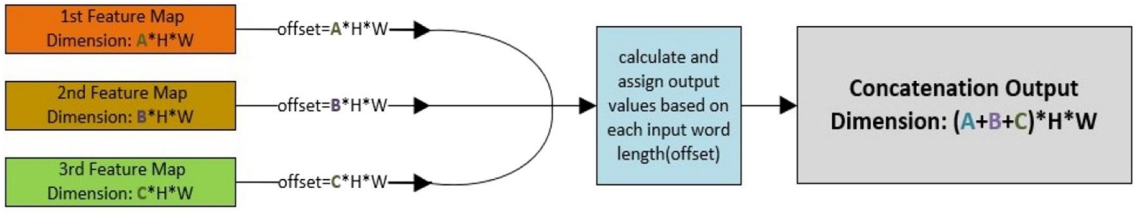

Fig. 7 Concatenation layer with 3 inputs. Getting input feature maps and calculating output based on offset (length)

multiple layer outputs. This approach can result in write-after-read (WAR) or read-after-write (RAW) conflicts within the HLS-based implementation of certain loops within each layer. Nonetheless, the reuse of these tensors can substantially reduce the memory requirements of the final design. Furthermore, our results suggest that it can lead to a slight improvement in latency.

The network's learned parameters are constant values that just play as a multiplication/addition factor in the design. HLS compiler will place them as read-only memory (ROM). Learned parameters have 32-bit IEEE floating-point precision. This amount of precision for storing in FPGA is not acceptable due to the limited resources of the PL part of the system-on-chip (SoC). Vitis-HLS has a library named *ap_fixed.h* that can be used to place floating-point values in FPGA with user-defined data type precisions. Using this library enables arbitrary precision datatypes within HLS design and can optimize HLS design in terms of resource usages like BRAM.

8 Implementation

The network incorporates two types of pooling layers, namely those with zero padding and those with non-zero padding. The Algorithm1(a) is applied to the pooling layers with zero padding, while it serves as a fundamental implementation for the non-zero padding layers, with conditions outlined in step 9. Within each filter element of the max-pooling layers, a criterion must be established to be compared with feature map.

The FX_MAX value is defined as the maximum value for our *ap_fixed* datatype, which is utilized in the calculations. For all the elements in the layer's filter, a comparison is conducted between the element and the local maximum value in the filter. The maximum value in the filter is then assigned as the corresponding output feature map value. As mentioned before, the output layer shape is calculated according to Eq. (2).

Algorithm 1(a): Basic pooling layer HLS implementation for YOLOv7-tiny
0. **Assume** S = stride=1, P = padding, F = (f_h, f_w)
1: **Inputs**: input feature map with dimension [x][y][z]
2: **Output**: output feature map with dimension [b][c][d]
3: define a max function for ap_fixed datatype
4: $for\ x_i\ in\ range(x)$:
5. set *local_max* variable to FX_MAX
6. $for\ y_i\ in\ range(y-P, y)$:
7. $for\ z_i\ in\ range(z-P, z)$:
8. iterate over each element in (f_h, f_w)
9. if the $z_i\ or\ y_i$ are out-of-bound then continue to the next iteration
10. else: $out[x_i][f_h+y_i][f_w+z_i] =$ max $(out[x_i][f_h+y_i][f_w+z_i], local_max)$

However, the implementation of non-zero-padding pooling layers cannot be carried out using Algorithm1(a). As a solution to this issue, we have introduced a method for implementing such layers in Algorithm1(b). In Algorithm1(b), all the iteration operations within the layer's filter are inverted to address the problem of out-of-bound indices that arises due to non-zero padding.

In Algorithm1(b), we have incorporated the *abs()* function from the *hls_math* library to validate indices within the layer's filter loops. A comparison between Algorithm1(a) and Algorithm1(b) was conducted by implementing both in our network within the HLS environment. The results revealed that Algorithm1(b) exhibited 50× less latency compared to Algorithm1(a) and consumed the same amount of memory for storing feature map values.

One of the primary contributions of Algorithm1(b) is the approach to iterate over the feature map values without the need for conditions to validate indices. Additionally, for pooling layers with zero padding values, step 9 of Algorithm1(a) is not required. This approach reduces the runtime of the Algorithm1(b) significantly compared to the basic Algorithm1(a). Moreover, non-zero padding values typically arise in convolutional layers for feature extraction purposes, and hence the behavior of Algorithm1(b) can also be employed in convolutional layers.

Defining the max function with the ternary operator in C++ (*condition ? a:b*) is not possible in HLS as this operator is not overloaded for arbitrary-precision datatypes. We can overload the question mark operator for arbitrary precision datatypes and then use a max macro. As we compared these approaches, defining a max function with *if condition* will synthesize a better RTL in terms of both latency and LUT usage.

9 Design optimizations

There are several levels of optimization to be done in the HLS project. All of the operations in coding a neural network in the HLS environment have to deal with iterations(loops). There are several types of optimizations to be done while iterating over loops. First, you have to know whether the loop you are working on is perfect (a nested loop with operations only on the innermost loop) or not.

For example, the code for convolutional layers applies 2 operations in one iteration on the outer loop. One for adding biases of the previous layer and another one for multiplication of the corresponded weights. The optimization of convolutional layers deals with loops that include operations. When pipelining these loops, it is preferable to pipeline bias loops if an initiation interval (II) of 1 can be achieved. This is due to the fact that bias loops of convolutional layers only contain assignment operations,

which require minimal resources, such as DSPs, compared to other solutions involving unrolling. Conversely, for computational loops such as the multiplication of input feature maps with layer weights, it is recommended to unroll with small factors to reduce the latency in the loop computation.

Algorithm 1(b): Optimized pooling layer HLS implementation for YOLOv7-tiny

0. **Assume** S = stride=1, P = padding, Filter = (f_h, f_w)
1: **Inputs**: input feature map with dimension [x][y][z]
2: **Output**: output feature map with dimension [b][c][d]
3: define a max function for ap_fixed datatype
4. set *local_max* variable to FX_MAX
5: $for\ x_i\ in\ range(x)$:
6. $for\ y_i\ in\ range(0, y)$:
7. $for\ z_i\ in\ range(0, z)$:
8. iterate over filter values of $[-f_x + 1,1], [-f_y + 1,1]$
9. $out[x_i][abs(f_h + y_i)][abs(f_w + z_i)] = out[x_i][abs(f_h + y_i)][abs(f_w + z_i)], local_max)$

Pooling layer optimizations can be done on the outermost loop, but there is a trade-off here between resource usage and latency; in a way that when you want to unroll the outer loop, it takes way more resource and consequently more power compared to unrolling the inner loops. But the benefit of unrolling outer loops is that it has better latency.

As we want our application to be real-time, so picking latency as our merit is the better choice. Here as we do the operations in two loops, unrolling these two loops with custom loop size factor is the best optimization. If there is no limited resource usage, the outermost loop that iterates over each feature map can also be unrolled with a factor of a quarter of the loop size because all the calculation for each slice of the feature map is independent of the other one and it's the best trade-off between resource usage and latency.

In the Leaky-Relu activation layer, there is a condition in the operation that checks if the value is less than 0 to do a multiplication with the $\alpha = 0.1$. So, the unrolling of the innermost loop makes an efficient optimization. Note that unrolling of the loops in all layers with different factors is not best optimization choice. There is a pragma in the Vitis HLS project that allows the HLS compiler to pipeline loops whose trip count is greater than the *threshold* [28] option to meet the timing requirements of the design concerning user-defined optimizations. Setting pipeline style to FRP (free-running flushable pipelining) results in the better delay requirement and reduces pipeline control fanout. Besides, FRP leads to more power consumption as the pipeline registers are clocked even if there is no data. According to [28], using the *config_array_partition* command tells the HLS compiler to partition arrays in the design in whatever dimension to reduce resource usage and speed up the design. The other optimization that improves resource usage and latency is using arbitrary precision data types in Vitis HLS. Results for using different datatype precisions while synthesizing all layers of the network are in Table 4.

Table 4 displays the results of the BRAM and LUT utilization for the entire network. Altering the data precision for all layers of the network can potentially reduce the usage of BRAM, allowing them to be set through AXI4 interfaces. Upon conducting experiments for various segments of the network, as well as for the entire network, discrepancies in BRAM and LUT usage were consistently observed. This may be attributed to the way HLS utilizes BRAMs for storing middle layer feature maps or how LUT allocations are performed based on calculations and assignments in each layer.

The reason for using *ap_fixed < 13,3 >* precision for numerical operations inside the HLS design is that by examining the entire model file, the biggest positive number absolute value was not greater than 2. Allocating 3 bits for the exponent part is enough (1 bit is reserved for the sign). Because of the multiplication of weights, it is possible that the exponent part is bigger than 1. So, allocating 2 bits for the exponent removes the chance of accuracy reduction in each feature map value. The more precision you put for the mantissa is better, but there is also a trade-off here that tells where to stop extending the mantissa because of limited resources on PL. Also allocating this amount of precision for network parameters has the minimum loss of accuracy among other datatypes in Table 4.

Synthesizing our design with other datatype precisions for exponent results in better resource utilization but according to our experiments it causes an accuracy reduction in final class predictions and probabilities. Although there are applications that these precisions for numerical operations is not obligatory. For example, BNN (binarized neural network) implementation could be accomplished using *ap_int* [28] types with a much smaller memory footprint.

According to the Vitis HLS compiler, it is recommended to avoid recursive calls to a function and refrain from calling a function inside another function. This leads to a huge design effort and imposes high time-complexity to implement every layer of the neural network.

Table 4 Comparison of different arbitrary precision data types resource usage in Vitis-HLS

Precision	LUT	BRAM
ap_fixed < 10,2 >	55 k	768
ap_fixed < 10,4 >	58 k	940
ap_fixed < 13,3 >	59 k	1092

Annotations in the output image might not be required. There is a small latency to annotate the output image with bounding boxes and class probabilities. But if the application just decides based on whether there is a specific object (e.g. cat) is in the image, then there is no need to annotate and we just need to pass a Boolean value to the next stage.

10 Results

The implementation results are shown in Table 5. The latency of our synthesized design in the HLS environment can satisfy real-time object detection requirements. Selection of the target device is based on how many resources are available in the PL part of the SoC. The hardware acceleration of the YOLOv7-tiny algorithm can be implemented on any Xilinx FPGA that can meet the requirements of PL resource usage. In this work, we've exported hardware for some of the zynq family FPGAs. Vitis-HLS tool is used for the synthesis and implementation of object detection algorithm on SoC achieving resource utilization of 59 K of Look-up tables, 9 K of Flip-flops, 1092 BRAMs and 16 DSPs for XC7Z100 with 13-bit datatype precision on XC7Z100 which is a good result based on the size of the network and its memory requirements. Achieving ~ 15 ms latency in synthesis results shows the ability of the design to be implemented in real-time environments.

The overall performance of our design with XC7Z7100 can satisfy real-time object detection applications. Approximately 9 K flip-flop usage is very low usage compared to all other designs. Also, LUT usage is lower than the average LUT usage of other works. Handling the control signals for our design takes only 16 DSP units which is much lower than other competitive works. The only high resource usage is BRAM which is better than some of the other works. According to [16], their best performance was @30fps input for FPGA. Our latency can also run at 30 fps input rate and higher input rates.

By employing a strategy of offloading memories from BRAM and setting them via DMI (direct memory interface) using AXI interface, it is possible to reduce BRAM usage and design for low-budget SoCs such as XC7Z7020 or XC7Z7045. Due to the latencies associated with data transfer from DMI, this approach may result in increased latencies compared to those acquired in our experiments with the XC7Z7100 board. Nevertheless, even with this new strategy on XC7Z7045, the design can meet real-time requirements with a latency of approximately 32 ms in the synthesized design. The work employs two tensors represented as separate C++ structs to capture network parameters and output feature maps. To enable the accelerator to operate on the XC7Z7045, certain arrays in these tensors must be deallocated from BRAM and set via the AXI4 interface for optimal model performance. This implementation on XC7Z7045 changes both FF and LUT usage a little bit because of control signals and rearrangement of new tensor arrays in BRAM.

According to Table 4, all SoCs except [16] possess either an equal or greater quantity of resources. For instance, in [13], the utilization of all resources is considerably greater than our design, and their network structure is several orders of magnitude simpler. Furthermore, Nakahara et al. [29] suggests a lightweight implementation of the YOLOv2 network on an FPGA with the ZCU102 board, which is considerably more expensive than ours, and their network is also less intricate. However, both accelerators in [13, 29] require more resources than our accelerator. Considering the smaller number of neural network parameters for scaled versions of YOLOv2 and YOLOv3 in [11, 13, 29, 30], along with their corresponding resource usage, it can be concluded that our proposed approach for YOLOv7-tiny on the XC7Z7100 with a much more complex network structure and a vast number of parameters can be implemented for real-time applications on low-budget SoCs such as XC7Z7045, as well as on the XC7Z7100 using a different strategy for memory accesses on BRAM. In Table 4, the accelerators have been designed to cater to various network topologies with varying memory requirements. In contrast to previous work [11, 13, 29–31], our proposed methods have

Table 5 Object detection results on FPGAs

Platform	Network	LUT	FF	BRAM	DSP	Clock	Power	GOP/s	Latency	
Zynq-7035	FPGA YOLO	47 k	40 k	787	409	N/A	7.518	N/A	–	[30]
ZCU102	Lightweight YOLOv2	135 k	370 k	1706	377	N/A	4.5	N/A	–	[29]
VC707	Sim-YOLOv2	155 k	115 k	1144	272	200	18.29	1877	339	[13]
Zedboard	YOLOv3-tiny	25.9 k	46.7 k	185	160	666.7	3.36	10.45	532	[11]
XC7Z7045	Angel-Eye	182.6 k	127.6 k	486	780	150	9.63	187.80	–	[31]
XC7Z7020	YOLOv4	27 k	24 k	68	198	100	10.36	189.14	18	[16]
XC7Z7020	YOLOv4	23.2 k	45.8 k	115	174	100	N/A	N/A	18	[16]
XC7Z7100	YOLOv7-tiny	59 k	9 k	1092	16	100	10.79	N/A	15	This work
XC7Z7045	YOLOv7-tiny	56 k	10 k	522	16	100	8.92	N/A	32	This work

successfully achieved real-time embedded application execution on two different SoCs. Specifically, our first method, as presented in this paper, can effortlessly operate the network at a frame rate of 30 frames per second on XC7Z7100. On the other hand, our second method, which employs HLS for accelerator design evaluation on a low-budget SoC such as XC7Z7045 and utilizes a DMI module, incurs additional latency in the final design. Despite this, it can still satisfy real-time inference requirements of 30 FPS or 24 FPS, rendering it suitable for a wide range of applications. Comparing our results with [16], our method consumes much lower DSP blocks in the FPGA and also less Flip-Flop usage and the final latency of our design is also lower.

By exporting our accelerator as an intellectual property (IP) to Vivado 2022.2 and measuring the power consumption, we have observed that power consumption in both designs utilizing XC7Z7100 and XC7Z7045 is competitive. However, in the case of the XC7Z7100 design, power consumption is mainly due to the significant usage of BRAM, which leads to higher power consumption in the PL. On the other hand, in our experiment with the XC7Z7045 design, we have achieved better power efficiency by utilizing a DMI instance and transferring data through AXI interface with fewer BRAM usage. Despite the fact that the XC7Z7045 design has enough space to function in real-time with input of at least 24 fps, we have determined that according to the SoC resources, the final implementation could be based on both design latencies.

We present the evaluation of optimization techniques on the accuracy of the final model for XC7Z7045. The purpose of testing these techniques in the final design is to observe the impact on accuracy. Testing the network on MS-COCO[27] test dataset with the utilization of arbitrary precision datatypes for storing all network parameters, which results in a mere 1.1% decrease in the final prediction tensor for objective scores compared to 32-bit floating-point design. This decrease is negligible compared to the gains achieved through embedded implementation and real-time inference. Furthermore, the techniques used to observe ONNX model properties using the *onnxruntime* library, have had a positive effect on the design process, preserving more of the original model accuracy compared to the methods without identifying network's configurations.

11 Conclusion

The hardware acceleration of convolutional neural networks can be a challenging task to optimize the implementation to meet the application's real-time requirement. Finding optimal values for optimizations including HLS-defined pragmas and algorithm optimizations of each layer of the network is time-consuming. In the proposed implementation, the main focus was to find an optimal point in terms of latency, resource usage, and also power consumption using different pragma parameters and various compile options in HLS.

According to recent research, disparities exist in the synthesis process of Vivado HLS and Vitis HLS. Given that Vitis HLS is a relatively new technology, it implements distinct pragmas and directives in contrast to its predecessor, Vivado HLS. Consequently, the design considerations for Vitis HLS differ from those applicable to the Vivado HLS compiler. Plenty of FPGA neural network implementations only target convolutional PE design like [32]. The convolutional layer is considered the most critical layer in a CNN, both in terms of computational complexity and resource utilization. But there are also considerations that are totally different between layers like pipelining types. Consequently, a way to design a CNN accelerator is to implement a PE for convolution operation. All other computational layer's PEs in the network should be implemented separately according to their specifications in the network and their required optimization techniques.

The exploration of probabilities pertaining to the execution of various network functions that can be accomplished within the PS of a SoC represents a promising avenue for future research endeavors into neural network accelerators in FPGAs. Nevertheless, advancements in neural network object detectors, such as the YOLO algorithm, have the potential to impact the implementation of such accelerators. For instance, the substitution of CNNs with BNNs for a particular object detector necessitates the development of specific PE implementations within the HLS design. The design of novel HLS technologies has the potential to reduce the development time of accelerators by examining various approaches for implementing each layer within the network and identifying the optimal method. This strategy can be viewed as an important consideration in the development of such technologies.

Author contributions AH: methodology, data curation, writing—original draft, visualization, software, validation.HJ: supervision, conceptualization, methodology, writing—review and editing, software.

Funding The authors did not receive support from any organization for the submitted work.

Data availability The datasets generated and/or analyzed during the present study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. He, K., Zhang, X., Ren, S., Sun, J.: Spatial pyramid pooling in deep convolutional networks for visual recognition. IEEE Trans. Pattern Anal. Mach. Intell. **37**(9), 1904–1916 (2015)
2. Girshick, R. Fast r-cnn. In Proceedings of the IEEE International Conference on Computer Vision, pp. 1440–1448. (2015).
3. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: towards real-time object detection with region proposal networks. Adv. Neural Inf. Process. Syst. **28** (2015).
4. Shawahna, A., Sait, S.M., El-Maleh, A.: FPGA-based accelerators of deep learning networks for learning and classification: a review. IEEE Access **7**, 7823–7859 (2018)
5. Dias, M.A., Ferreira, D.A.: Deep learning in reconfigurable hardware: a survey. In 2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 95–98. IEEE, (2019).
6. El-Shafie, A.H.A., Habib, S.E.: Survey on hardware implementations of visual object trackers. IET Image Proc. **13**(6), 863–876 (2019)
7. Wang, J., Lin, J., Wang, Z.: Efficient hardware architectures for the deep convolutional neural network. IEEE Trans. Circuits Syst. I Regul. Pap. **65**(6), 1941–1953 (2017)
8. Babu, P., Parthasarathy, E.: Hardware acceleration of image and video processing on Xilinx Zynq platform. Intell. Autom. Soft Comput. **30**(3) (2021).
9. Pestana, D., Miranda, P.R., Lopes, J.D., Duarte, R.P., Véstias, M.P., Neto, H.C., De Sousa, J.T.: A full-featured configurable accelerator for object detection with YOLO. IEEE Access **9**, 75864–75877 (2021)
10. Babu, P., Parthasarathy, E.: Optimized object detection method for FPGA implementation. In 2021 Sixth International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET), pp. 72–74. IEEE, (2021).
11. Nguyen, D.T., Nguyen, T.N., Kim, H., Lee, H.J.: A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection. IEEE Trans. Very Large Scale Integr. (VLSI) Syst. **27**(8), 1861–1873 (2019)
12. Zeng, K., Ma, Q., Wu, J.W., Chen, Z., Shen, T., Yan, C.: FPGA-based accelerator for object detection: a comprehensive survey. J. Supercomput. **78**(12), 14096–14136 (2022)
13. Yap, J.W., bin Mohd Yussof, Z., bin Salim, S.I., Lim, KC.: Fixed point implementation of tiny-YOLO-v2 using OpenCL on FPGA. Int. J. Adv. Comput. Sci. Appl. **9**(10) (2018).
14. Günay, B., Okcu, S.B., Bilge, H.Ş. LPYOLO: low precision YOLO for face detection on FPGA. arXiv preprint arXiv:2207.10482 (2022).
15. Yu, Z., Bouganis, C.S. A parameterisable FPGA-tailored architecture for YOLOv3-tiny. In Applied ReconFigureurable Computing. Architectures, Tools, and Applications: 16th International Symposium, ARC 2020, Toledo, Spain, April 1–3, 2020, Proceedings 16, pp. 330–344. Springer International Publishing, (2020).
16. Babu, P., Parthasarathy, E.: Hardware acceleration for object detection using YOLOv4 algorithm on Xilinx Zynq platform. J. Real-Time Image Proc. **19**(5), 931–940 (2022)
17. Redmon, J., Farhadi, A. Yolov3: an incremental improvement. arXiv preprint arXiv:1804.02767 (2018).
18. Bochkovskiy, A., Wang, C.Y., Liao, H.Y. Yolov4: optimal speed and accuracy of object detection. arXiv preprint arXiv:2004.10934 (2020).
19. Wang, C.Y., Liao, H.Y., Wu, Y.H., Chen, P.Y., Hsieh, J.W., Yeh, I.H.: CSPNet: a new backbone that can enhance learning capability of CNN. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, pp. 390–391. (2020).
20. Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4700–4708. (2017).
21. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7132–7141. (2018).
22. Woo, S., Park, J., Lee, J.Y., Kweon, I.S.: Cbam: Convolutional block attention module. In Proceedings of the European Conference on Computer Vision (ECCV), pp. 3–19. (2018).
23. Liu, S., Qi, L., Qin, H., Shi, J., Jia, J. Path aggregation network for instance segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 8759–8768. (2018).
24. Wang, C.Y., Bochkovskiy, A., Liao, H.Y.M. YOLOv7: trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. arXiv preprint arXiv:2207.02696 (2022).
25. Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2117–2125. (2017).
26. Long, X., Deng, K., Wang, G., Zhang, Y., Dang, Q., Gao, Y., Shen, H., Ren, J., Han, S., Ding, E., Wen, S.: PP-YOLO: an effective and efficient implementation of object detector. arXiv preprint arXiv:2007.12099 (2020).
27. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, CL.: Microsoft coco: common objects in context. In Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13, pp. 740–755. Springer International Publishing, (2014).
28. Xilinx. Vitis High-Level Synthesis User Guide. (2022).
29. Nakahara, H., Yonekawa, H., Fujii, T., Sato, S.: A lightweight YOLOv2: A binarized CNN with a parallel support vector regression for an FPGA. In Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, pp. 31–40. (2018).
30. Wei, G., Hou, Y., Cui, Q., Deng, G., Tao, X., Yao, Y.: YOLO acceleration using FPGA architecture. In 2018 IEEE/CIC International Conference on Communications in China (ICCC), pp. 734–735. IEEE, (2018).
31. Guo, K., Sui, L., Qiu, J., Yao, S., Han, S., Wang, Y., Yang, H.: Angel-eye: a complete design flow for mapping cnn onto customized hardware. In 2016 IEEE Computer Society Annual Symposium on VLSI (ISVLSI), pp. 24–29. IEEE, (2016).
32. Nguyen, X.Q., Pham-Quoc, C.: An fpga-based convolution ip core for deep neural networks acceleration. REV J. Electron. Commun. **12**(1–2) (2022).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.